System Design in a Hurry

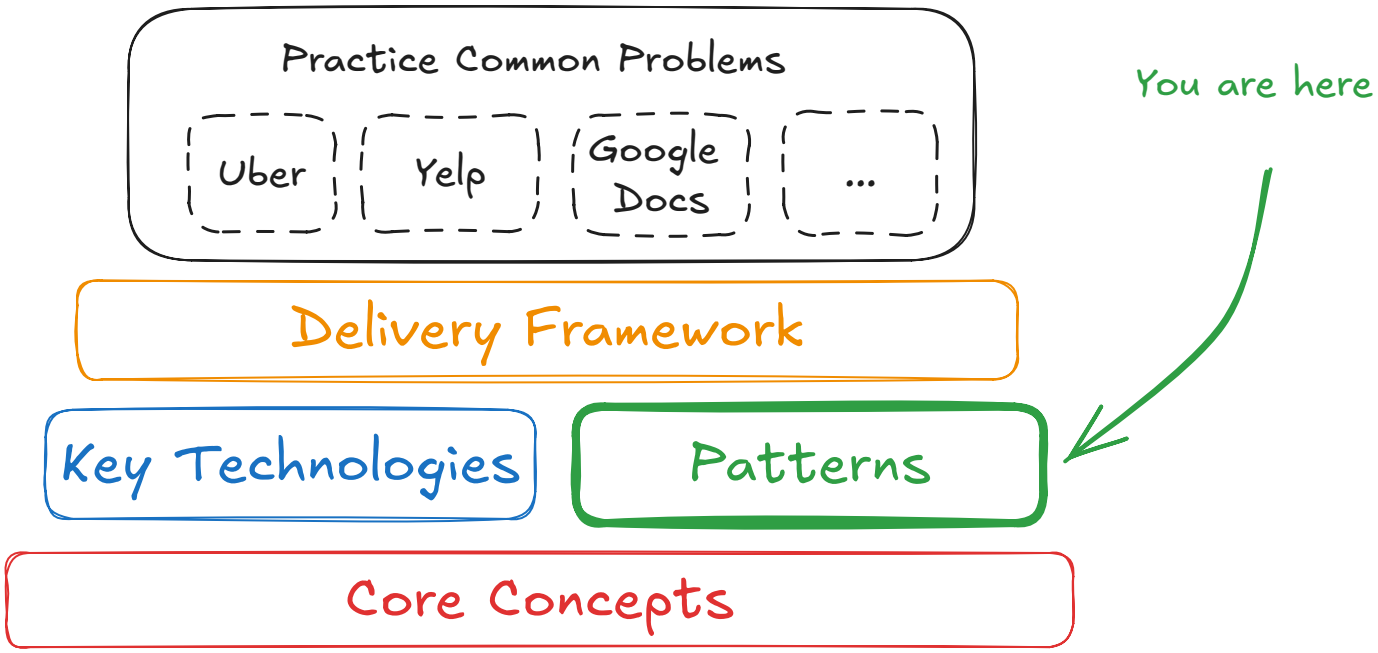
Common Patterns

The most common system design interview patterns, built by FAANG managers and staff engineers

Updated Jul 18, 2025

By taking the **key technologies** and **core concepts** we've discussed and combining them, you can build a wide variety of systems. But success in the time-constrained environment of interviewing is all about patterns. If you're able to identify the patterns that are required for a specific design, not only can you fall back to best practices but you'll save a bunch of time trying to reinvent the wheel.

> The ability to identify and apply patterns is a skill that often separates senior engineers from more junior engineers in the system design interview. Patterns allow you to know what's interesting and what's not, they also save you time by helping you to see common failure modes rather than reverse engineering them on the fly!



Overall Structure

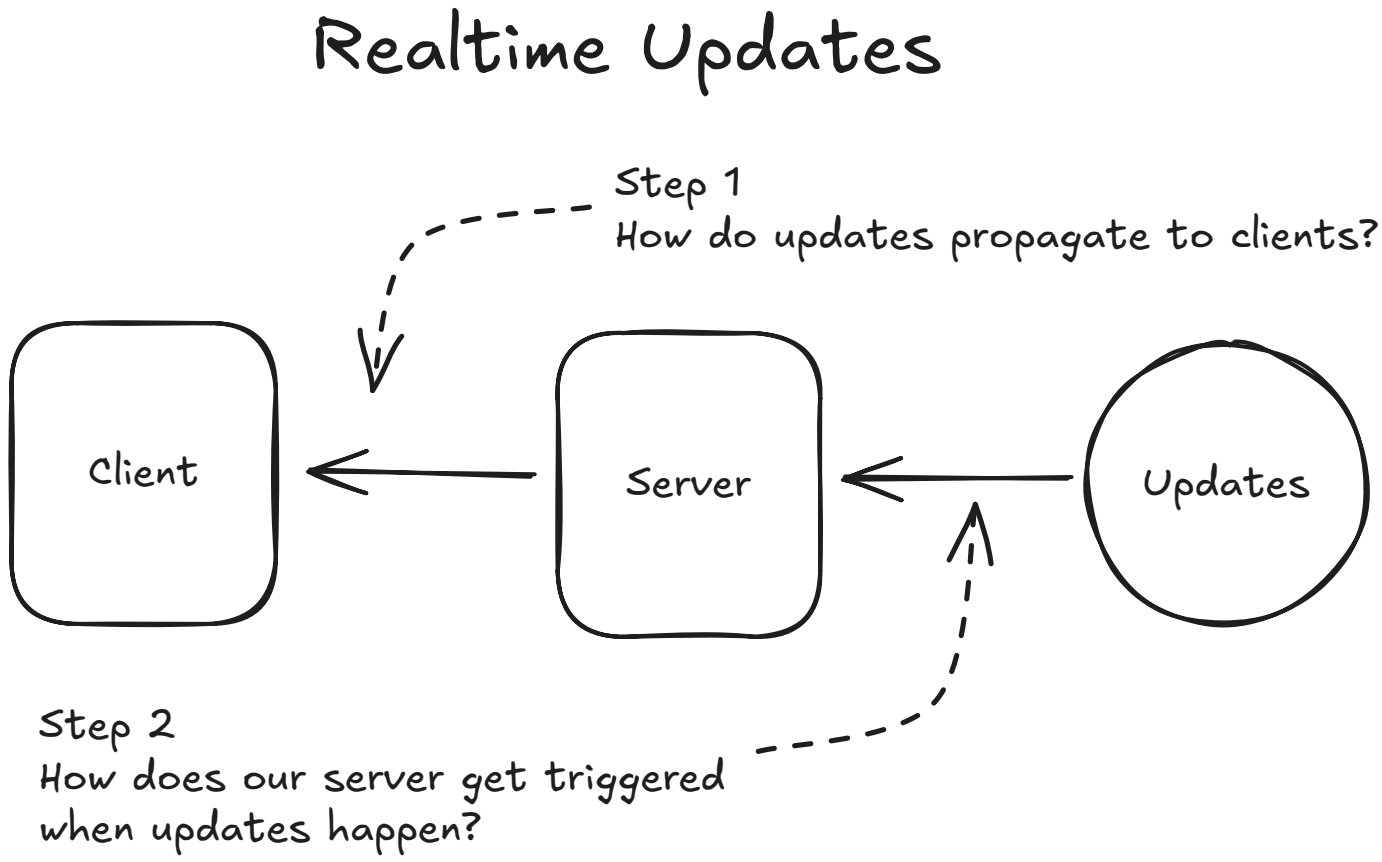
What follows are some common patterns that you can use to build systems. These patterns are not mutually exclusive, and you'll often find yourself utilizing several of them to build a system. In each of our **Problem Breakdowns**, we'll call out patterns that are used to build the system so you can spot these commonalities and read about common deep dives and pitfalls.

Pushing Realtime Updates

In many systems, you'll need to be able to make updates to the user in real-time. For synchronous APIs, this is as simple as returning a response once the request is completed. For other systems like chat applications, notifications, or live dashboards, you'll need to be able to push updates to the user as they happen.

There are a lot of decisions to make when implementing realtime updates. First, you'll need to choose a protocol. Simple HTTP polling is the simplest option, but it's not the most efficient. Server-sent events (SSE) and websockets are purpose-built for realtime updates, but the infrastructure can be tricky to get right. Read our **Networking Essentials** core concept for a deep dive on the protocol choice. We generally recommend starting with HTTP polling until it no longer serves your needs. Once you're there, you can consider SSE or websockets.

For the server side of realtime updates, you again have more options! Pub/Sub services are a common way to decouple the publisher and subscriber (used in our **Whatsapp** breakdown), while stateful servers in a consistent hash ring or other configuration can be used for situations where processing is heavier (used in our **Google Docs** breakdown).



Realtime Updates Challenges

We talk about all of these options at length in our **Pushing Realtime Updates** Pattern.

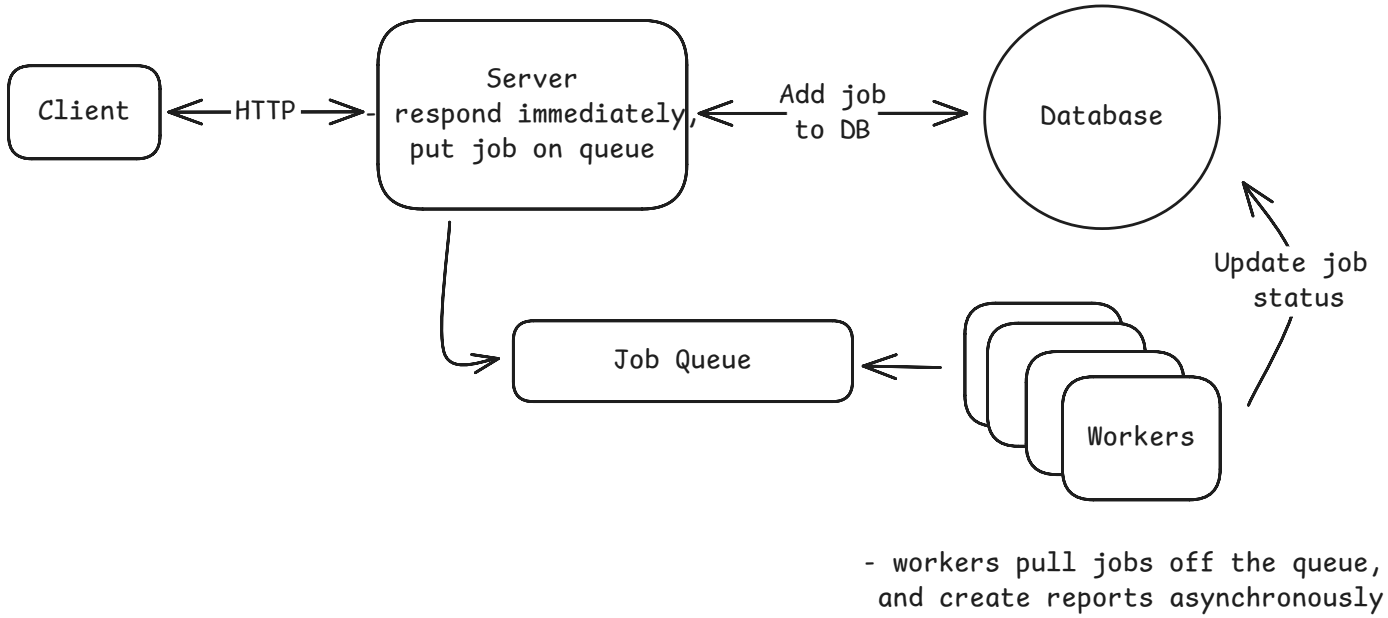
Managing Long-Running Tasks

Many operations in distributed systems take too long for synchronous processing - video encoding, report generation, bulk operations, or any task that takes more than a few seconds. The Managing Long-Running Tasks pattern splits these operations into immediate acknowledgment and background processing.

When users submit heavy tasks, your web server instantly validates the request, pushes a job to a queue (like Redis or Kafka), and returns a job ID within milliseconds. Separate worker processes continuously pull jobs from the queue and execute the actual work. This provides fast user response times, independent scaling of web servers and workers, and fault isolation.

> Many candidates are quick to pull the trigger on pushing their processing behind a queue, but this is frequently a bad decision and you need to be careful about the tradeoffs. If you have short-running jobs, returning the status of the job synchronously with the request simplifies your architecture dramatically providing clearer back-pressure and better user experience.

The key technologies are message queues for job coordination and worker pools for processing. You'll need to handle job status tracking, retries, and failure scenarios like dead letter queues for poison messages.



Long Running Tasks

Get the full breakdown of async worker pools, job queues, and failure handling in our Managing Long-Running Tasks Pattern.

Dealing with Contention

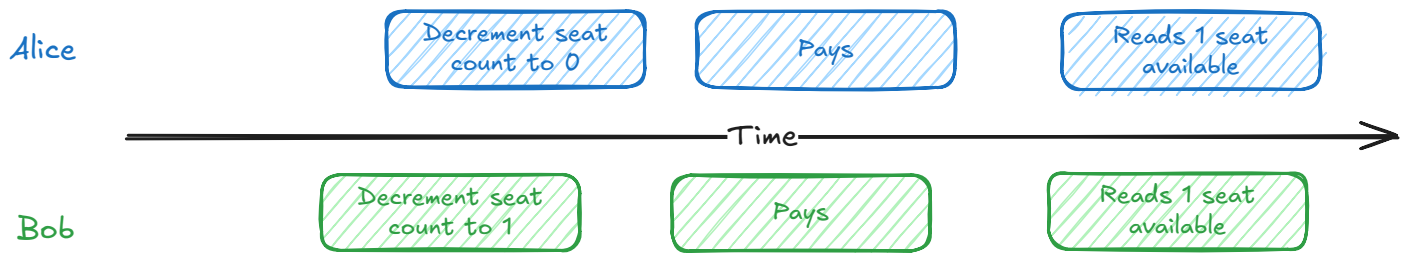
When multiple users try to access the same resource simultaneously, like booking the last concert ticket or bidding on an auction item, you need mechanisms to prevent race conditions and ensure data consistency. This pattern addresses coordination challenges in distributed systems.

Solutions range from database-level approaches like pessimistic locking and optimistic concurrency control to distributed coordination mechanisms. The key is understanding when to use atomicity and transactions versus explicit locking strategies. For distributed systems, you might need distributed locks, two-phase commit protocols, or queue-based serialization.

Trade-offs include performance versus consistency guarantees, and simple database solutions versus complex distributed coordination. Most problems start with single-database solutions before scaling to distributed approaches.

> Databases are built around problems of contention. When you separate your data into multiple databases, you're taking on all of the challenges that the database systems were originally designed to solve. In some cases this can be completely appropriate, but be

> careful about doing it prematurely. Interviewers are keen to dig in and see if you really understand all that you're giving up by breaking your data apart.



Example of a race condition

Dive deeper into locks, transactions, and distributed coordination techniques in our **Dealing with Contention** Pattern.

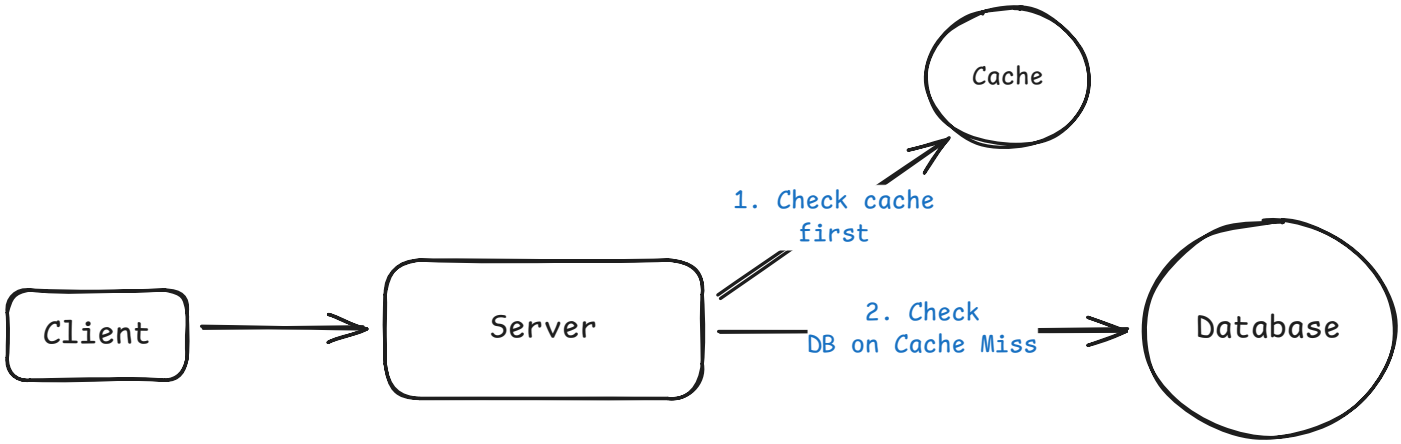
Scaling Reads

As your application grows from hundreds to millions of users, read traffic often becomes the first bottleneck. While writes create data, reads consume it - and read traffic typically grows much faster than write traffic. The Scaling Reads pattern addresses high-volume read requests through database optimization, horizontal scaling, and intelligent caching.

For most applications, the read-to-write ratio starts at 10:1 but often reaches 100:1 or higher. Consider Instagram: when you open the app, you see dozens of photos requiring hundreds of database queries for metadata, user info, and engagement data. Meanwhile, you might only post once per day - a single write operation.

The solution follows a natural progression: optimize read performance within your database through indexing and denormalization, scale horizontally with read replicas, then add external caching layers like Redis and CDNs.

Key considerations include managing cache invalidation, handling replication lag in read replicas, and dealing with hot keys where millions of users request the same popular content simultaneously.



Database Read Scaling

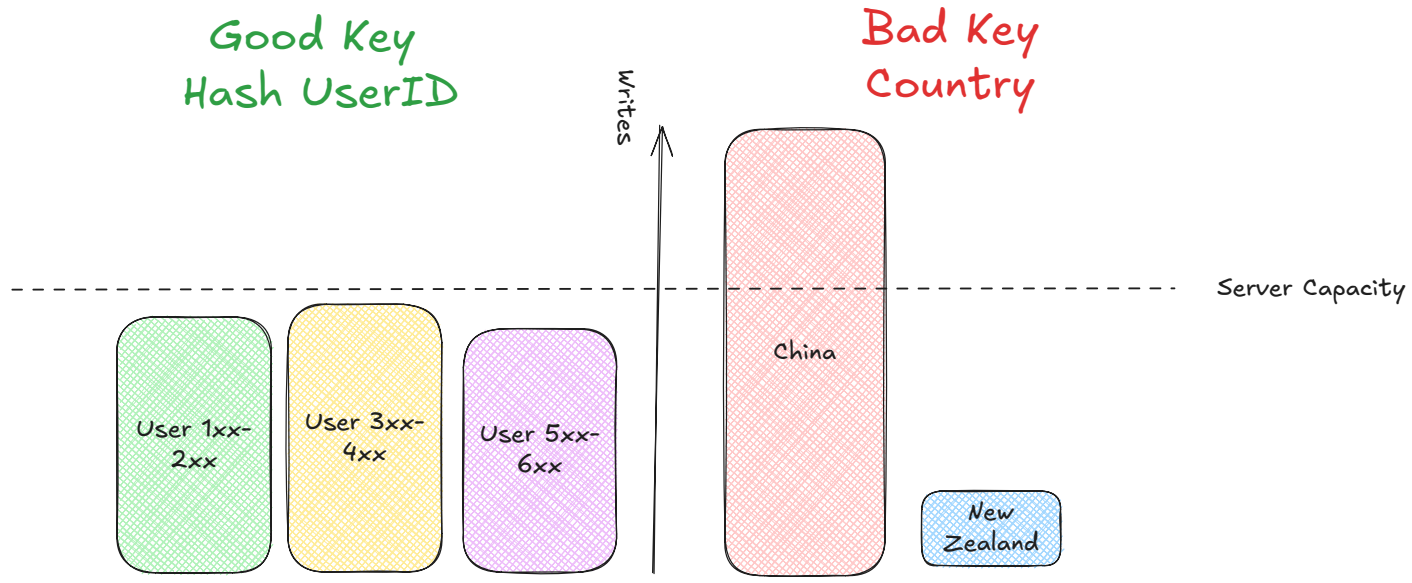
Learn about indexing strategies, read replicas, and cache invalidation patterns in our **Scaling Reads** Pattern.

Scaling Writes

As your application grows from hundreds to millions of writes per second, individual database servers and storage systems hit hard limits. The Scaling Writes pattern addresses write bottlenecks through **sharding**, batching, and intelligent load management.

The core strategies are horizontal **sharding** (distributing data across multiple servers), vertical partitioning (separating different types of data), and handling write bursts through queues and load shedding. Key considerations include selecting good partition keys that distribute load evenly while keeping related data together.

For burst handling, you can use write queues to buffer temporary spikes or implement load shedding to prioritize important writes during overload. Batching techniques help reduce per-operation overhead by grouping multiple writes together.



Good and Bad Partition Keys

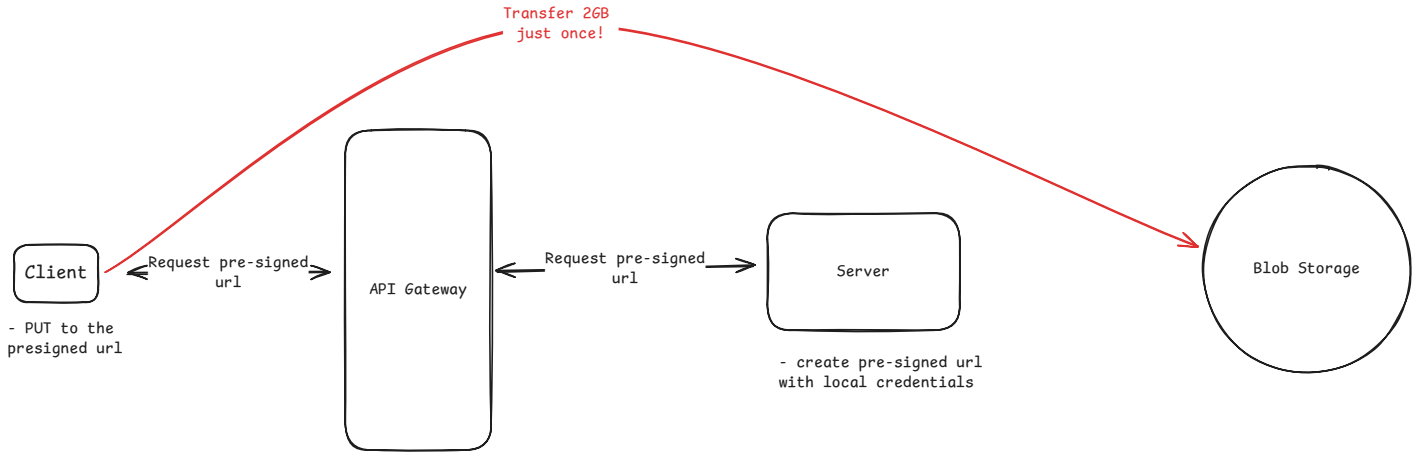
Read our comprehensive guide to **sharding**, partitioning, and handling write bursts in our **Scaling Writes** Pattern.

Handling Large Blobs

Large files like videos, images, and documents need special handling in distributed systems. Instead of routing gigabytes through your application servers, this pattern uses direct client-to-storage transfers with presigned URLs and CDN delivery.

Your application server generates temporary, scoped credentials (presigned URLs) that let clients upload directly to blob storage like S3. Downloads come from CDNs with signed URLs for access control. This eliminates your servers as bottlenecks while providing resumable uploads, progress tracking, and global distribution.

Key challenges include state synchronization between your database metadata and blob storage, handling upload failures, and managing the lifecycle of large files. Event notifications from storage services help keep your application state consistent.



Large Blobs

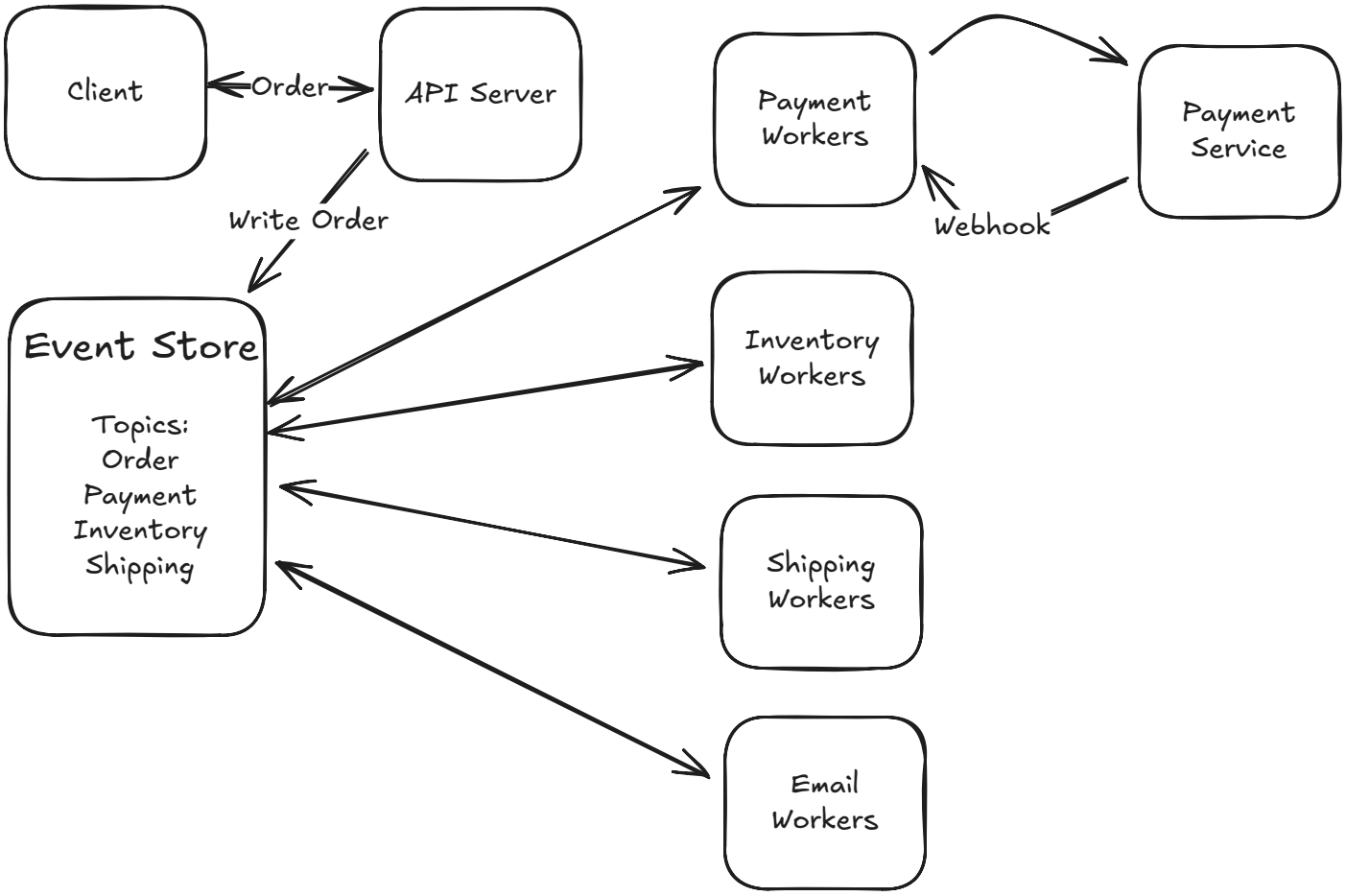
Explore advanced techniques for presigned URLs, resumable uploads, and CDN delivery in our **Large Blobs** Pattern.

Multi-Step Processes

Complex business processes often involve multiple services and long-running operations that must survive failures, retries, and external dependencies. This pattern provides reliable coordination for workflows like order fulfillment, user onboarding, or payment processing.

Solutions range from simple single-server orchestration to sophisticated workflow engines and durable execution systems. Event sourcing provides a distributed approach where each step emits events that trigger subsequent steps. Modern workflow systems like Temporal or AWS Step Functions handle state management, failure recovery, and retry logic automatically.

The key insight is moving from scattered state management and manual error handling to declarative workflow definitions where the system guarantees exactly-once execution and maintains complete audit trails.



Multi-Step Processes

See detailed examples and implementation strategies for workflow engines and durable execution in our **Multi-Step Processes** Pattern.

Proximity-Based Services

Several systems like **Design Uber** or **Design Gopuff** will require you to search for entities by location. **Geospatial indexes** are the key to efficiently querying and retrieving entities based on geographical proximity. These services often rely on extensions to commodity databases like **PostgreSQL with PostGIS extensions** or **Redis' geospatial data type**, or dedicated solutions like Elasticsearch with geo-queries enabled.

The architecture typically involves dividing the geographical area into manageable regions and indexing entities within these regions. This allows the system to quickly exclude vast areas that don't contain relevant entities, thereby reducing the search space significantly.

While geospatial indexes are great, they're only really necessary when you need to index hundreds of thousands or millions of items. If you need to search through a map of 1,000 items, you're better off scanning all of the items than the overhead of a purpose-built index or service.

Note that most systems won't require users to be querying globally. Often, when proximity is involved, it means users are looking for entities *local* to them.

Pattern Selection

These patterns often work together to solve complex system design challenges. A video platform might use **Large Blobs** for video uploads, **Long-Running Tasks** for transcoding, **Realtime Updates** for progress notifications, and **Multi-Step Processes** to coordinate the entire workflow.

The key is recognizing which patterns apply to your specific problem and understanding their trade-offs. Start with simpler approaches (polling, single-server orchestration) and only add complexity when you have specific requirements that demand it.

In system design interviews, proactively identifying and applying these patterns demonstrates architectural maturity and helps you focus on the most important aspects of your design rather than getting bogged down in implementation details.